

INTERNAL TOOLING · SEPTEMBER 2026

Two skills that turn a domain into a deck.

client-logo pulls a client's real brand out of their own site. client-deck turns it into an editable PowerPoint that always looks like the same company made it.

-
- 01 Name a domain
 - 02 client-logo pulls, cleans and animates the mark
 - 03 It writes a brand file
 - 04 client-deck builds the deck from that brand

`~/claude/skills/client-deck` · `~/claude/skills/client-logo`

client-deck

Name a client, hand over content, get a .pptx the sales team can edit.

The point is continuity. The design system lives in `brands/<client>.json` and `lib/deck-kit.mjs`, never in the deck script. Two decks for the same client, built months apart by different sessions, come out looking like the same company made them. A deck script holds strings and numbers. It holds no geometry.

Running it

```
# build, and animate on save
node build-deck.mjs

# one PNG per slide, for looking at it
pwsh -File ~/.claude/skills/client-deck/scripts/render-qa.ps1 \
  -Pptx out.pptx -OutDir tmp/render

# a PDF to send
pwsh -File ~/.claude/skills/client-deck/scripts/to-pdf.ps1 -Pptx out.pptx
```

All three scripts drive desktop PowerPoint over COM with no window, so nothing steals focus. There is no LibreOffice and no pdftoppm on this machine, so the pptx skill's soffice path does not apply here.

Animation

`deck.save()` adds a fade transition to every slide and a short content reveal in reading order. The effects are native PowerPoint objects, so the deck stays editable and works offline. Pass `{ animate: false }` when the destination needs a static deck, or `animationStyle: "transition-only"` when object reveals would be inappropriate. To animate a deck that already exists, without overwriting it:

```
pwsh -File ~/.claude/skills/client-deck/scripts/animate-pptx.ps1 \
  -Pptx in.pptx -OutPath out-animated.pptx
```

The kit

Everything is in inches. Every helper returns where its content ended.

CALL

WHAT IT DRAWS

<code>title()</code>	Full bleed opener, large logo, monogram motif
<code>section()</code>	Act divider with a giant ghosted numeral
<code>slide()</code>	Standard slide, with an optional edge panel and motif
<code>panel()</code>	Color region bled off one slide edge
<code>rows()</code>	Editorial list, hairline separated
<code>stat()</code> <code>statRow()</code>	One figure, or a row of them on shared baselines
<code>steps()</code>	Numbered process flow
<code>block()</code>	Eyebrow, head and copy on a flat fill
<code>caseStudy()</code>	A whole case study: band, hero figure, before and after
<code>priceCol()</code>	Pricing column, one of them accented
<code>table()</code>	Native PowerPoint table, brand styled
<code>flow()</code>	Auto height text that returns the next y
<code>notes()</code>	Speaker notes with a [Sources] block
<code>save()</code>	Writes the file, animates it, verifies it, prints fit warnings

CHAIN FLOW(), DO NOT HARDCODE Y

Every helper returns the y its content ended at. Passing that into the next call is what keeps a two line heading from landing on the paragraph under it. Pass bottom: `deck.BOT` to `rows`, `statRow`, `steps` and `bullets` so an overflow is caught at build time rather than in the render.

The visual language

Editorial, not cards on a grid.

What it produces

- Full bleed color panels that touch a slide edge, not boxes floating inside margins
- A wide type scale, 54pt display against 12.5pt body, so hierarchy does the work
- Figures set large against open space; the number is the hero and its label is small
- Hairline rules between rows and columns instead of borders and fills
- One motif: the brand monogram, oversized at about 90% transparency, bleeding off an edge
- A light and dark sandwich, with one or two fully dark content slides for rhythm

What it will not produce

- An accent line or rule under a slide title
- A decorative color bar, header stripe, sidebar stripe down a slide edge, or a single edge border on a block
- A row of three equal cards as a default layout
- Centered body copy. Titles and stat values may center; paragraphs and lists never do
- AI purple, neon glow, glassmorphism, floating gradient orbs
- Dashes in visible slide copy, including as a bullet glyph

Arial, deliberately

The deck has to render identically on a sales laptop that has never heard of the client's web font. The brand file records the real web font in `type.webFont` for reference, and the deck does not use it.

Text fitting

PowerPoint does the layout, so the kit estimates it. The line estimator models Arial's average advance width with separate constants for regular, bold, caps and bold caps. Bold caps run about fifteen percent wider than bold mixed case, and one constant for both leaves a visible gap under every label. The estimate errs toward reserving more room.

Where siblings have to line up, pass an explicit height rather than letting each measure itself. A one line head that estimates as two will otherwise sit lower than the block beside it.

Two failure modes worth knowing

Both cost real time the first time they happen.

POWERPOINT REFUSES THE FILE AND NOTHING ELSE DOES

A text box with height zero or less makes PowerPoint refuse to open the whole presentation, while python-pptx opens it happily, the XSD passes it, and the pptx skill's **validate.py** reports "All validations PASSED".

If PowerPoint says it could not open the file and every other tool says the file is fine, hunt for a negative height shape. The kit now clamps and warns instead.

A DARK PANEL DOES NOT MAKE THE SLIDE DARK

rows, statRow, steps, bullets and block each take their own dark: `true`. Miss it and you get ink on navy, which renders as an invisible line of text that no validator will catch.

If a slide is mostly one ground, make the whole slide that ground and pass dark everywhere, rather than layering a panel over a light slide.

The sourcing rule

Keep a `SOURCES.md` beside the deck with one row per statistic, result and price, naming the page it came from. A case study with no hard number gets presented qualitatively. Never write a plausible sounding metric. Anything unverified goes into an explicit sign off list for the client.

`notes()` writes the sources into the speaker notes, so whoever is presenting can defend any number on screen.

client-logo

Name a domain, get that client's real logo, cleaned properly, with motion.

```
node ~/.claude/skills/client-logo/logo.mjs kit needmomentum.com \
  --out ./brand --brand momentum-digital
```

That pulls, renders, animates and writes a brand file client-deck can load. The four stages also run on their own.

COMMAND	WHAT IT DOES
pull <domain>	Find and download logo candidates, with provenance
render <file>	Trim, knock out, recolor, split, resize, contrast check
animate <dir>	MP4, transparent WebM, GIF and CSS
kit <domain>	All three, plus the brand file

PROVENANCE IS THE WHOLE POINT
Assets come from the client's own domain and nowhere else. Every file lands in **provenance.json** next to the URL it came from and why that candidate scored highest.
No third party logo APIs. They serve stale marks, pre rebrand marks, and sometimes a different company's mark entirely. Shipping the wrong logo to a prospect is worse than shipping no logo.

Most real sites block the direct fetch

Agency and small business sites sit behind SiteGround, Cloudflare or similar, which answer a scripted request with a challenge page. pull detects that and says so, rather than parsing two hundred bytes of meta refresh as a homepage. Three ways through:

- Open the site with the browser tool, save the HTML, then pull <domain> --html page.html. Discovery runs on the saved markup while downloads still go direct, which usually works because asset hosts are less guarded than page routes.
- If the asset host is guarded too, fetch the bytes inside the browser, write them to disk, and run render on that file. Skip pull entirely.
- If the logo URL is already known, pull <domain> --asset <url>.

What rendering properly means

The work that separates a usable asset from a screenshot of one.

STEP	WHY IT MATTERS
Background knockout	If all four corners agree on one opaque color, it is removed with a tolerance, so a JPEG or flattened PNG comes back transparent
True trim	Cropped to the ink bounding box, not to whatever padding the file shipped with. Baked in whitespace sits wrong in every layout it lands in
Color versions	Full color, white knockout and dark ink, each preserving antialiased edges rather than hard thresholding them
Lockup split	Finds the monogram and wordmark by comparing the widest column gap against the median of the others, so it splits a real lockup and leaves an evenly spaced wordmark alone
Palette	Ink colors ranked by frequency weighted by saturation, so the primary is the brand color and not an antialiasing artifact. Reports null for an achromatic mark instead of inventing one
Contrast	WCAG ratios against white and against dark navy, with a warning naming which version to use where
Sizes	2400, 1200, 600 and 300px wide, re rasterized from the vector where there is one, so nothing is upscaled

WHY THE CONTRAST CHECK EARNS ITS PLACE

18.73:1

Stripe's mark on white

Perfectly legible

1.04:1

The same mark on dark navy

Effectively invisible

This is the case where a knockout is not optional, and it is not obvious by eye until the slide is already built.

Motion

Three stings, each about two seconds, each built from the client's own palette. No glow, no orbs, no bounce.

rise The mark scales up a few percent and settles as a brand color rule draws underneath it. Default

wipe A brand color bar sweeps across and leaves the mark behind it

split The monogram lands, then the wordmark slides out from behind it. Needs a lockup split

Each writes an MP4 for decks and social, a WebM with real transparency for overlaying on video, a GIF for email and chat, and a CSS only HTML file with the logo inlined that honors prefers-reduced-motion and needs no video file at all. Ground defaults to a deep tint of the brand color; - - light gives white.

Chaining them

kit writes the brand file in the shape client-deck expects.

```
cp brand/<id>.brand.json ~/.claude/skills/client-deck/brands/<id>.json
mkdir -p ~/.claude/skills/client-deck/assets/<id>
cp brand/logo*.png brand/mark*.png \
  ~/.claude/skills/client-deck/assets/<id>/
```

The palette is derived from the real logo, and aspect and markAspect are measured rather than guessed. Set name, contact and titlewidth by hand.

ONE STANDING CAVEAT
Always check the derived palette against the client's own brand guide before anything goes to a prospect. Colors sampled from a compressed PNG are a good starting point, not an authority. **kit** prints this reminder for the same reason.

Requirements

Node and pptxgenjs	Installed inside client-deck, so a deck script anywhere on disk can import the kit by absolute path
sharp	Installed inside client-logo. Rasterizes SVG, which is what makes vector logos usable
ffmpeg	On PATH, for animate only. pull, render and the CSS output work without it
Desktop PowerPoint	For animate-pptx, render-qa and to-pdf, all driven over COM with no window

Where things live

~/.claude/skills/client-deck/	SKILL.md, lib/deck-kit.mjs, brands/, assets/, scripts/
~/.claude/skills/client-logo/	SKILL.md, logo.mjs
Both are hand authored	Versioned in the .claude repo, unlike the marketplace installs alongside them